

Dedicated VXLAN capacity: what limits us?

Decision brief • Actual measurements, not a throughput projection

Bare interface creation exceeds 1,000/s. The complete Nullnet path does not reach that target in these tests.

BARE CREATE

4,301–5,530/s

Per host; interfaces UP; unique UDP ports

ENCRYPTED SETUP

19.7/s

1,000 offered clients; expiry disabled; profiled

SETUP + CHURN

11.1/s

One-second idle timeout; unmodified binaries

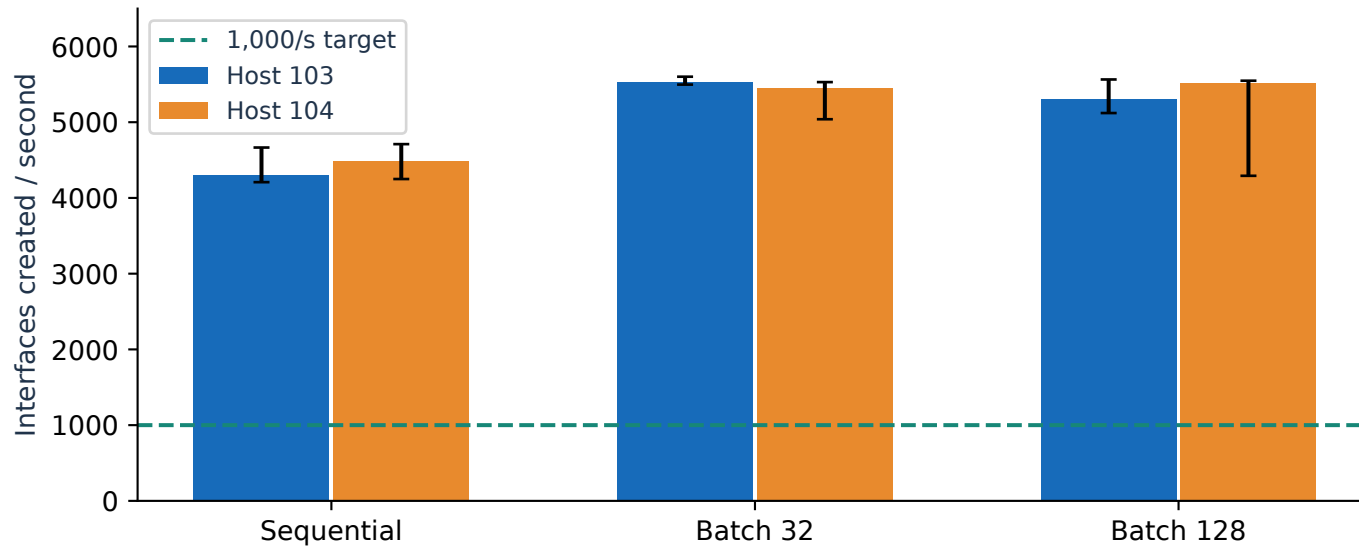
1. Creation batching helps bare throughput, but a missing bulk-create API is not the main explanation for the full-system gap.
2. Retiring 1,000 unique-port VXLANs takes about 20 seconds. Sharing port 4789 cuts deletion to ~0.19 seconds (page 6).
3. The stage profile identifies where Nullnet waits; those wall times include kernel contention, subprocess execution, and scheduling. They are not CPU-time attribution.

All primary runs returned HTTP 200: creation-only 1000/1,000; churn 1000/1,000. A 1,000/s offered burst is a stress input, not achieved capacity. Results apply to this two-node lab and short test duration.

The kernel can create quickly; deletion changes the picture

Bare devices in isolated network namespaces • 1,000 devices per run • Three repeats per batch size

Creation: median and observed range

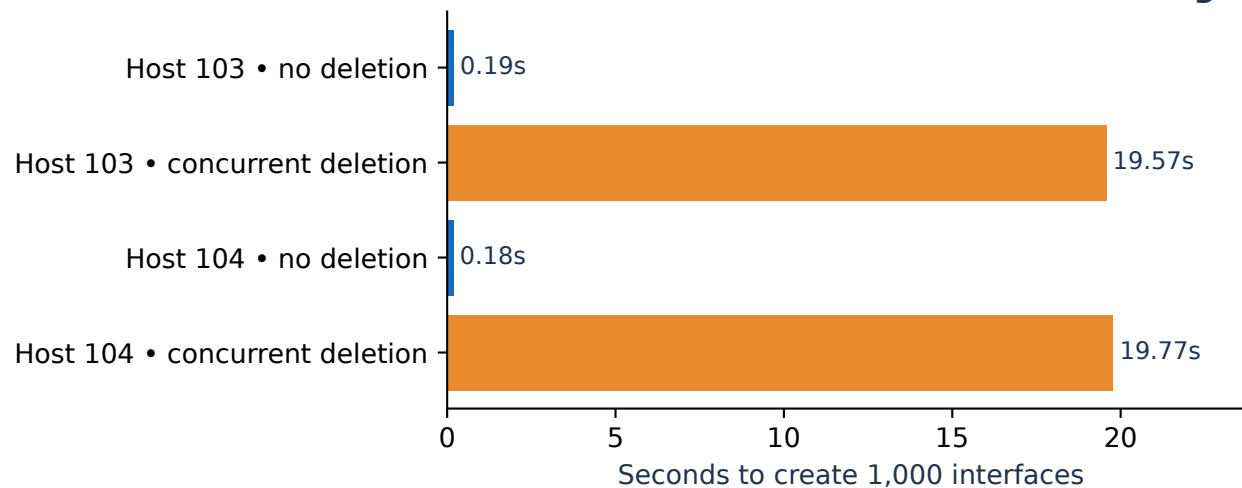


Continuous create/delete cycles

Host 103: 51.0 cycles/s
Median delete: 19.53s per 1,000 devices

Host 104: 49.0 cycles/s
Median delete: 20.05s per 1,000 devices

Creation while another cohort is retiring

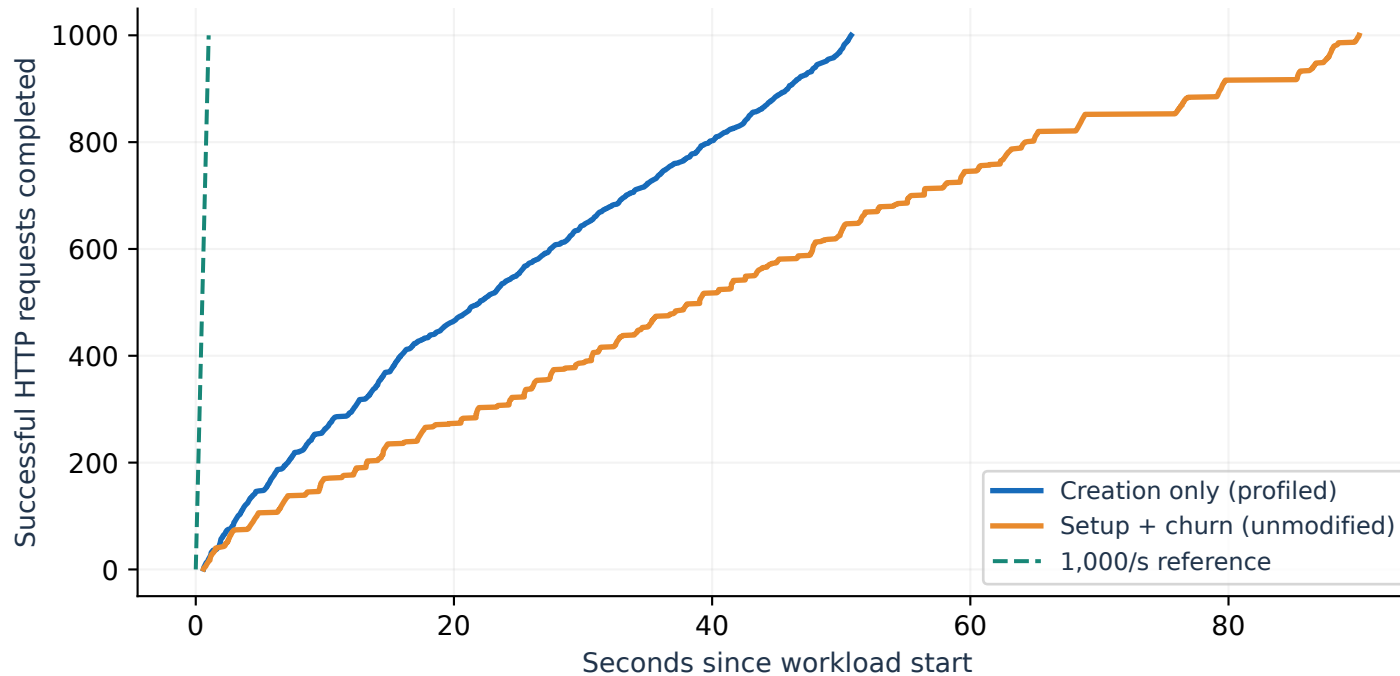


All bare runs and overlap tests had zero Netlink errors and zero residual VXLAN devices. Nine create/delete cohorts per host were measured. This is a bounded churn test, not a long-duration soak.

Each device had its own UDP port, matching Nullnet's encrypted cross-host allocation. An unencrypted dummy underlay was used; no traffic traversed these bare devices.

Complete encrypted connections: observed readiness

1,000 distinct source IPs offered at 1,000/s • max_networks = NULL • All service replicas on host 103



Creation only
50.77s to complete
19.70 ready/s
Mean: 22.80s
p99: 49.56s
Errors: 0

Setup + churn
90.11s to complete
11.10 ready/s
Mean: 39.87s
p99: 88.90s
Errors: 0

Readiness here means HTTP 200 through the real proxy and encrypted network. It includes routing admission/control work, endpoint setup, and the application response. The request path is "/"; backend fanout application work is not included. Each source gets a dedicated ingress network. The creation-only snapshot verified 1,000 live graph edges and 2,000 encrypted XFRM states on each host.

Host CPU during unmodified churn (normalized across 8 CPUs): 103: mean 39.0%, peak 79.3%; 104: mean 39.1%, peak 79.5%. The moderate average does not exclude a serialized bottleneck. The diagnostic churn repeat achieved 12.51/s; it is kept separate from the primary capacity result.

Where endpoint setup time is spent

Mean elapsed milliseconds per endpoint • Cross-host VXLAN endpoints only • Timings include waiting

	Create only 103: container	Create only 104: proxy	Churn 103: container	Churn 104: proxy
Per-network lock	0.0	0.0	0.0	0.0
Namespace creation / Docker PID lookup	32.1	31.4	95.3	214.3
Stale veth lookup / cleanup	0.7	92.4	430.5	86.1
Veth creation into target namespace	1.7	83.1	194.8	85.8
Namespace address / UP / route	21.7	70.4	109.8	223.1
Bridge lookup / cleanup / creation	2.2	182.0	174.0	167.2
Bridge address / MTU / UP	5.1	257.2	239.9	220.5
Veth lookup / bridge attachment	1.9	159.4	131.8	121.5
Cross-host stale-interface checks / cleanup	1.7	311.7	155.7	175.4
VXLAN creation	0.8	84.7	51.4	45.1
VXLAN lookup / bridge attach / UP	1.8	162.0	113.4	104.7
IPsec key derivation / states / policies	41.6	45.1	138.0	266.8
Forwarding sysctl / iptables	93.9	114.5	276.5	333.2

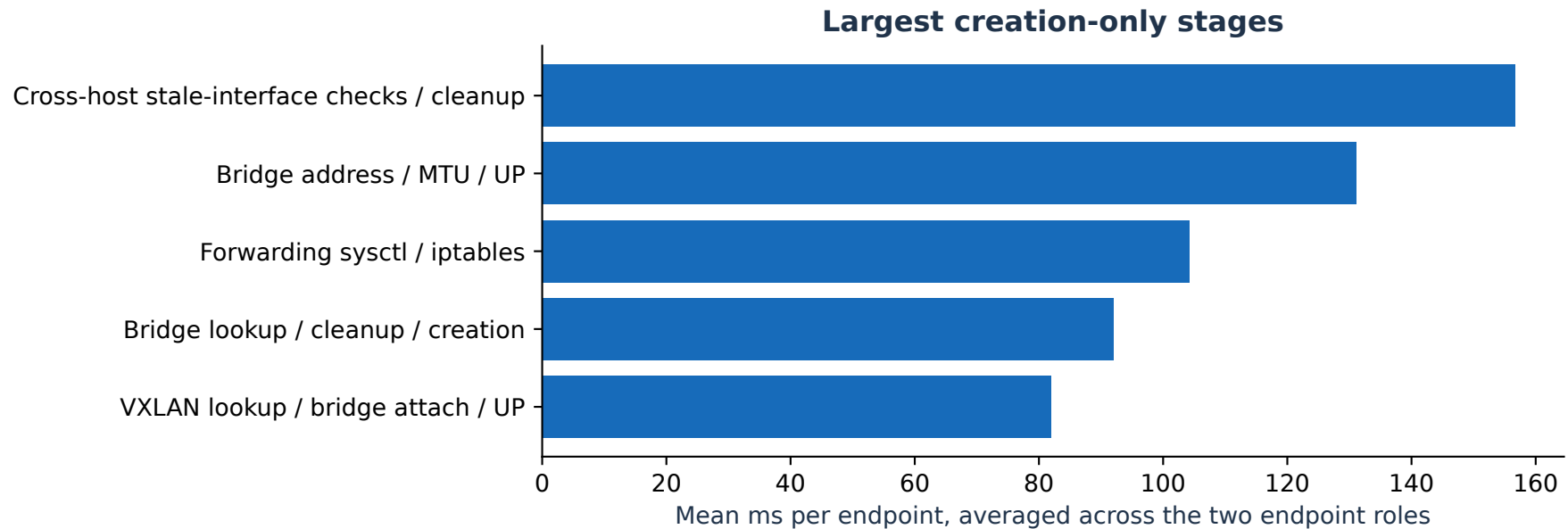
Mean endpoint totals (columns left to right): 205.2 ms, 1,593.9 ms, 2,111.1 ms, 2,043.9 ms.

Included subprocess-admission wait: 67.2 ms, 82.4 ms, 290.6 ms, 525.3 ms. This is a subset of the stage totals; do not add it again.

Endpoint stages are sequential; endpoints and clients overlap. Do not sum endpoint averages to explain HTTP latency. Stale-interface stages include existence checks even when nothing is deleted.

Interpretation, coverage, and next decision

The profile identifies expensive stages; it does not turn elapsed waiting into kernel CPU attribution



Measured conclusions

- Bare unique-interface creation clears the 1,000/s target on both hosts.
- Deleting old unique-port devices severely delays concurrent new-device creation.
- Full setup includes many operations beyond RTM_NEWLINK; the table shows their cost.
- Faster batching alone is not evidence that the full system can sustain 1,000 new connections/s.

Scope and limits

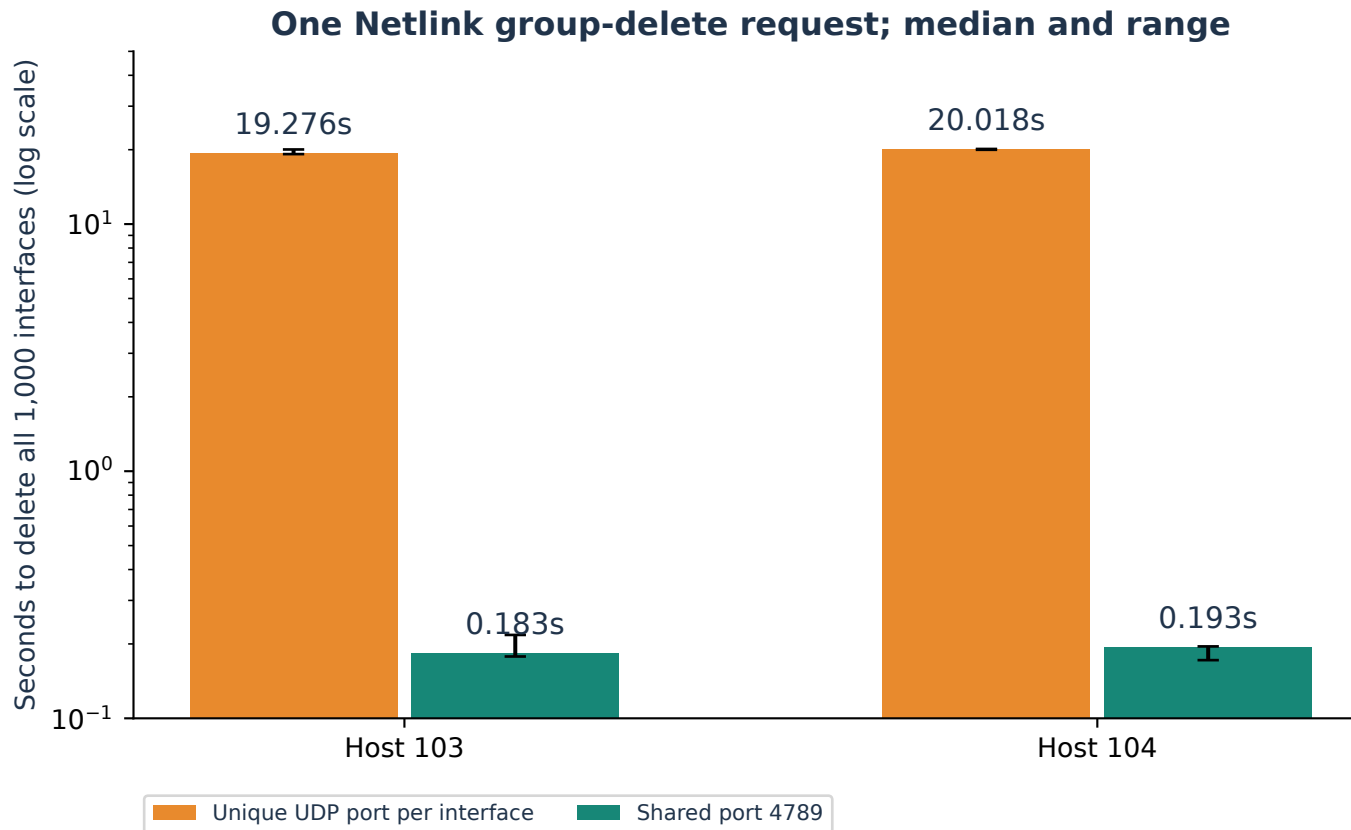
Linux 6.12.95+deb13-amd64; 8 logical CPUs and approximately 16 GB RAM per host. Baseline binaries are from merged #199; the temporary client probe adds timings only. Full setup uses TLS control, IPsec-encrypted cross-host VXLAN, and 12 service containers. Bare tests use isolated namespaces. No 60,000-active-interface capacity or long-duration stability claim is made.

Valid endpoint records (103 / 104): creation only 997 / 998; churn 993 / 990, out of 1,000 expected each. Total malformed records excluded: 22.

Endpoint-only profile: proxy admission, server/storage and transport are not individually timed. The HTTP-to-endpoint gap remains unattributed.

Shared UDP port: about 100× faster deletion

Matched A/B • 1,000 unique VXLAN interfaces and VNIs in both cases • Batch 32 • Three repeats per host



Same interface count.
Same unique VNIs.
Only the UDP port allocation changed.

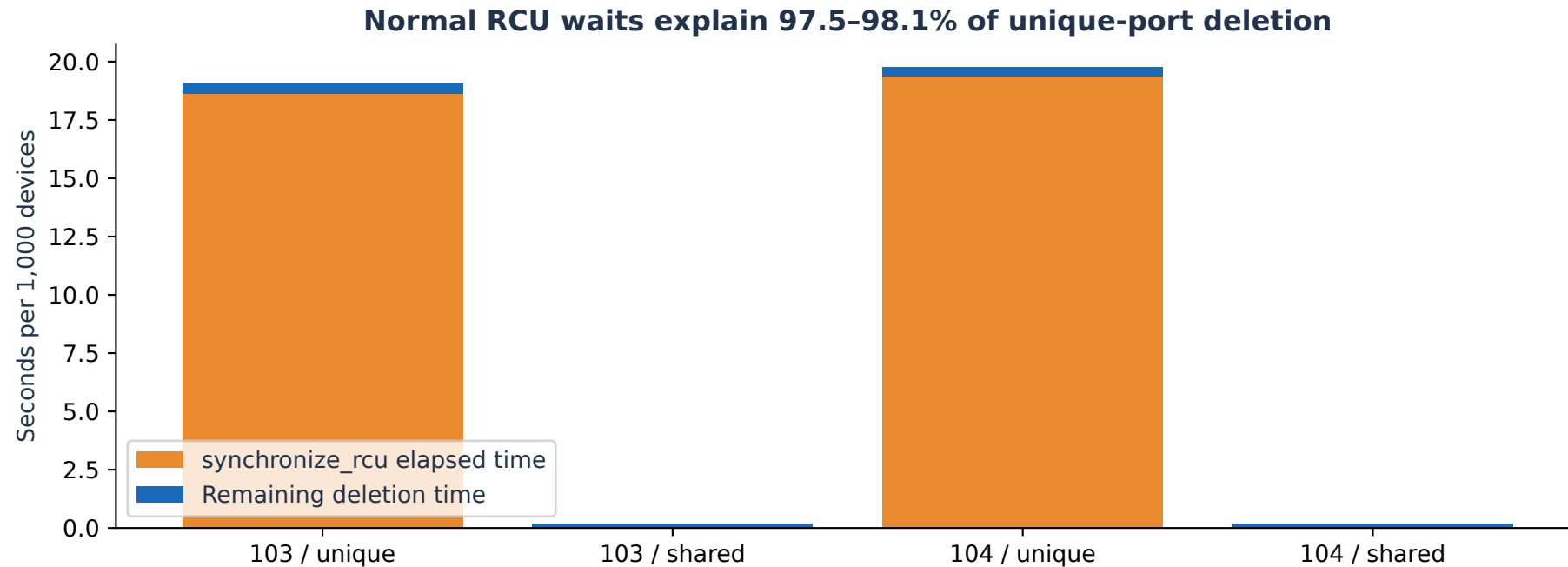
Shared-port creation also remained fast: approximately 5,700–5,900 interfaces/s. Every paired run had zero errors and zero residual devices.

The earlier ~20-second result is specific to the unique-port configuration. It is not an unavoidable cost of deleting 1,000 distinct VXLAN interfaces. Kernel tracing now confirms serial RCU grace-period waits in UDP socket release as the dominant cost (page 7).

Integration is not yet validated: Nullnet currently uses unique UDP ports to distinguish per-tunnel IPsec policies. Sharing the VXLAN port while preserving the required encryption and isolation needs a separate design and end-to-end test. These bare tests carried no traffic and installed no IPsec state.

Kernel cause confirmed: one RCU wait per UDP socket

Function-graph tracing on both lab kernels • One additional 1,000-device run per configuration



Unique ports: 1,000 UDP socket releases and 1,000 normal RCU waits. Total RCU wait: 18.616s on 103 and 19.363s on 104. Shared port: one socket release and one normal RCU wait (17.7ms / 23.1ms). All 1,000 distinct interfaces are still released.

Kernel path: `vxlan_sock_release` → last-reference check → `udp_tunnel_sock_release` → `synchronize_rcu`. This waits for existing RCU readers before freeing socket state. The per-interface `synchronize_net` uses expedited RCU while RTNL is held; that faster path still runs with a shared port. Group deletion does not batch the normal socket-release waits.

This explains teardown, not creation-only setup latency. Complete Nullnet readiness was 50.8s for 1,000 requests (~20/s). Endpoint stage times include queueing; proxy/server/storage/transport and execution versus kernel-wait attribution remain incomplete. Tracing instances and test namespaces were removed; no product changes.